Manifesto for Agile Software Development



We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler
James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick
Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

Agile Overview

- Agile project management is based on an incremental, iterative approach.
  - Instead of in-depth planning at the beginning of the project, Agile methodologies are open to changing requirements over time and encourages constant feedback from the end users.
- The **goal of each iteration** is to produce a **working product**.
- Agile refers to **any process that aligns with the concepts of the Agile Manifesto**.
- In 2001, 17 software developers met to discuss lightweight development methods. They published the Manifesto for Agile Software Development, which covered how they found “better ways of developing software by doing it and helping others do it.”

What is “Agility”?

- Effective (rapid and adaptive) response to change (team members, new technology, requirements)
- Effective communication in structure and attitudes among all team members, technological and business people, software engineers and managers.
- Drawing the customer into the team. Eliminate “us and them” attitude. Planning in an uncertain world has its limits and plan must be flexible.
- Organizing a team so that it is in control of the work performed.
- Eliminate all but the most essential work products and keep them lean.
- Emphasize an incremental delivery strategy as opposed to intermediate products that gets working software to the customer as rapidly as feasible.

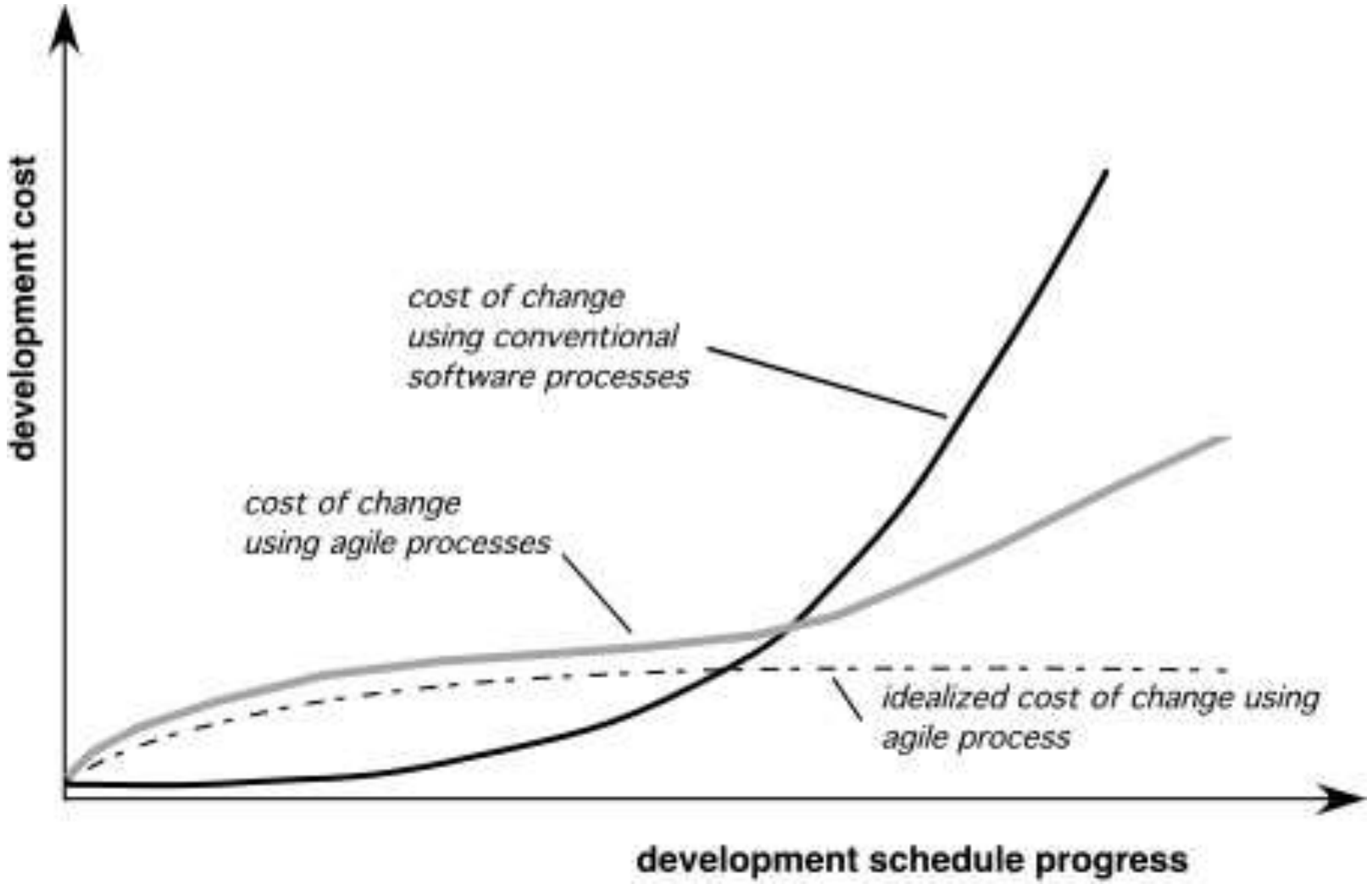
Why and What Steps are“Agility” important?

Why? The modern business environment is fast-paced and ever-changing. It represents a reasonable alternative to conventional software engineering for certain classes of software projects. It has been demonstrated to deliver successful systems quickly.

What? May be termed as “software engineering lite” The basic activities- communication, planning, modeling, construction and deployment remain. But they morph into a minimal task set that push the team toward construction and delivery sooner.

The only really important work product is an operational “software increment” that is delivered.

Agility and the Cost of Change



An Agile Process

- Is driven by customer descriptions of what is required (scenarios).
- Some assumptions:
  - ✓ Recognizes that plans are short-lived (some requirements will persist, some will change. Customer priorities will change)
  - ✓ Develops software iteratively with a heavy emphasis on construction activities (design and construction are interleaved, hard to say how much design is necessary before construction. Design models are proven as they are created.)

  Analysis, design, construction and testing are not predictable.
- Thus has to Adapt as changes occur due to unpredictability
- Delivers multiple ‘software increments’, deliver an operational prototype or portion of an OS to collect customer feedback for adaption.

12 Agile Principles :

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face–to–face conversation.

Agile Principles cont…

7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from self–organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Advantages of Agile

- Change is embraced: With shorter planning cycles, there's always opportunity to refine and reprioritize the backlog to accommodate changes throughout the project.
- End-goal can be unknown: Agile is beneficial for projects where the end-goal is not clearly defined. As the project progresses, the goals will become evident and the team can adapt.
- Faster, high-quality delivery: Breaking down the project into iterations allows the team to focus on high-quality development, testing, and collaboration. Conducting testing during each iteration means that bugs get identified and solved more quickly.

- Strong team interaction: Agile embraces frequent communication and face-to face interactions.
- Customers are heard: Customers have many opportunities to see the work being delivered, share their input, and have an impact on the end product.
- Continuous improvement: Feedback is encouraged from users and team members throughout the project, so lessons learned are used to improve future iterations

Disadvantages of Agile

- Planning can be less concrete: Because project managers are often reprioritizing tasks, it’s possible some items scheduled for delivery may not be complete in time. And, additional sprints may be added at any time in the project, adding to the overall timeline.
- Team must be knowledgeable: Agile teams are usually small, so team members must be highly skilled in a variety of areas and understand Agile methodology.

- Time commitment from developers: Active involvement and collaboration is required throughout the Agile process, which is more time consuming than a traditional approach.

- Documentation can be neglected: Agile prefers working deliverables over comprehensive documentation. While documentation on its own does not lead to success, teams should find the right balance between documentation and discussion.

Extreme Programming (XP)

- The most widely used agile process,
- It uses an object-oriented approach.
- Planning
- Design
- Coding
- Test

XP Planning

- Begins with the listening, leads to creation of “user stories” that describes required output, features, and functionality. Customer assigns a value (i.e., a priority) to each story.
- Agile team assesses each story and assigns a cost (development weeks. If more than 3 weeks, customer asked to split into smaller stories)
- Working together, stories are grouped for a deliverable increment next release.
- A commitment (stories to be included, delivery date and other project matters) is made. Three ways:
  - 1. Either all stories will be implemented in a few weeks, 2. high priority stories first, or 3. the riskiest stories will be implemented first.
- After the first increment “project velocity”, namely number of stories implemented during the first release is used to help define subsequent delivery dates for other increments. Customers can add stories, delete existing stories, change values of an existing story, split stories as development work proceeds

User Story



As an Account Manager
I want a sales report of my account to be sent to my inbox daily
So that I can monitor the sales progress of my customer portfolio

Acceptance criteria:

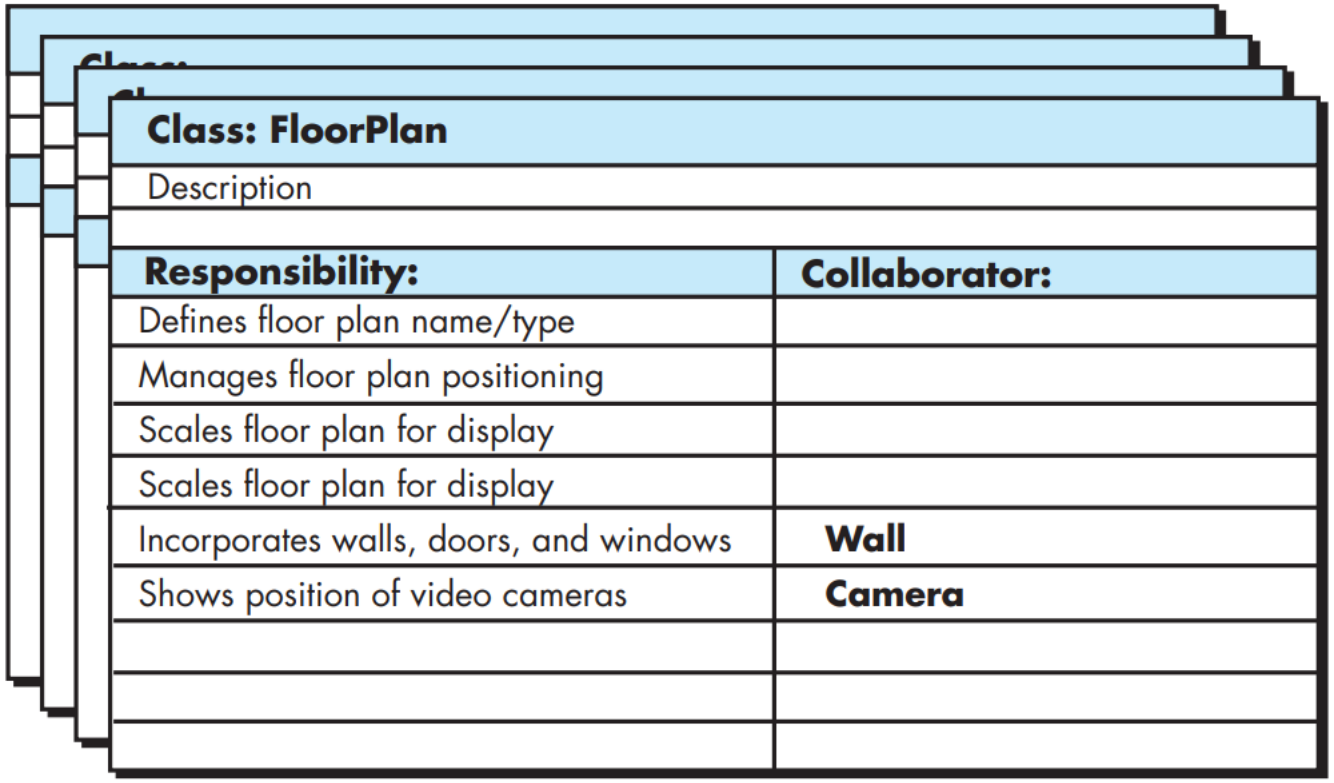
1. The report is sent daily to my inbox
2. The report contains the following sales details: …
3. The report is in csv format.

XP Design

- Occurs both before and after coding as refactoring is encouraged
  - Follows the KIS principle (keep it simple) Nothing more nothing less than the story.
  - Encourage the use of CRC (class-responsibility-collaborator) cards in an object-oriented context. The only design work product of XP. They identify and organize the classes that are relevant to the current software increment.
  - For difficult design problems, suggests the creation of “spike solutions”—a design prototype for that portion is implemented and evaluated.
  - Encourages “refactoring”—an iterative refinement of the internal program design. Does not alter the external behavior yet improve the internal structure. Minimize chances of bugs. More efficient, easy to read.

- A CRC model is really a collection of standard index cards that represent classes. The cards are divided into three sections. Along the top of the card you write the name of the class. In the body of the card you list the class responsibilities on the left and the collaborators on the right.

A CRC model index card



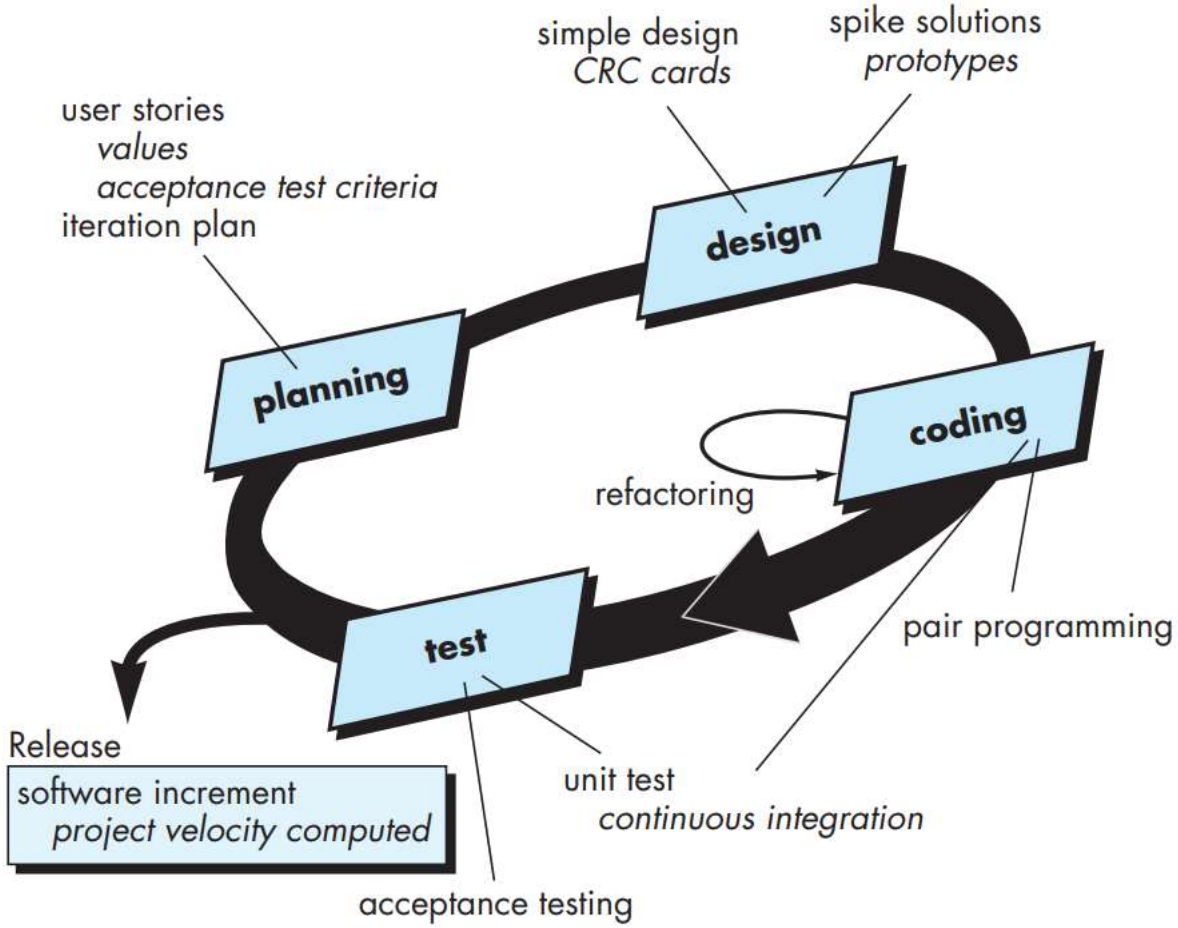
Class: FloorPlan	
Description	
Responsibility:	**Collaborator:**
Defines floor plan name/type	
Manages floor plan positioning	
Scales floor plan for display	
Scales floor plan for display	
Incorporates walls, doors, and windows	**Wall**
Shows position of video cameras	**Camera**

XP Coding and Testing

- XP Coding
  - Recommends the construction of a unit test for a story *before* coding commences. So implementer can focus on what must be implemented to pass the test.
  - Encourages “pair programming”. Two people work together at one workstation. Real time problem solving, real time review for quality assurance. Take slightly different roles.
- XP Testing

  All unit tests are executed daily and ideally should be automated. Regression tests are conducted to test current and previous components.
  - “Acceptance tests” are defined by the customer and executed to assess customer visible functionality

Extreme Programming (XP)



The XP Debate

- Requirements volatility: Customer is an active member of XP team, changes to requirements are requested informally and frequently.
- Conflicting customer needs: Different customers' needs need to be assimilated. Different vision or beyond their authority.
- Requirements are expressed informally: User stories and acceptance tests are the only explicit manifestation of requirements. Formal models may avoid inconsistencies and errors before the system is built. Proponents said changing nature makes such models obsolete as soon as they are developed.
- Lack of formal design: XP deemphasizes the need for architectural design. Complex systems need overall structure to exhibit quality and maintainability. Proponents said incremental nature limits complexity as simplicity is a core value.

Other Agile Process Models

- Adaptive Software Development
- Scrum
- Dynamic Systems Development Method(DSDM)
- Feature Driven Development(FDD)
- Agile Modeling
- and more…

Adaptive Software Development (ASD)

- Originally proposed by Jim Highsmith (2000) focusing on human collaboration and team self-organization as a technique to build complex software and system.
- ASD — distinguishing features
  - Mission-driven planning
  - Component-based focus
  - Uses “time-boxing” (timeline for each component with cost)
  - Explicit consideration of risks
  - Emphasizes collaboration for requirements gathering
  - Emphasizes “learning” throughout the process

ex: planning a college event

- You know the goal: make the event successful. This is **mission-driven planning**.
- Different groups handle music, food, decorations, and games. This is a **component-based focus**.
- Each group gets a deadline and budget. This is **time-boxing**.
- The team thinks early about problems, like rain or not enough money. This means considering **risks**.
- Everyone discusses what is needed and shares ideas. This is **collaboration**.
- After each step, the team checks what worked and improves the next step. This is **learning throughout the process**.

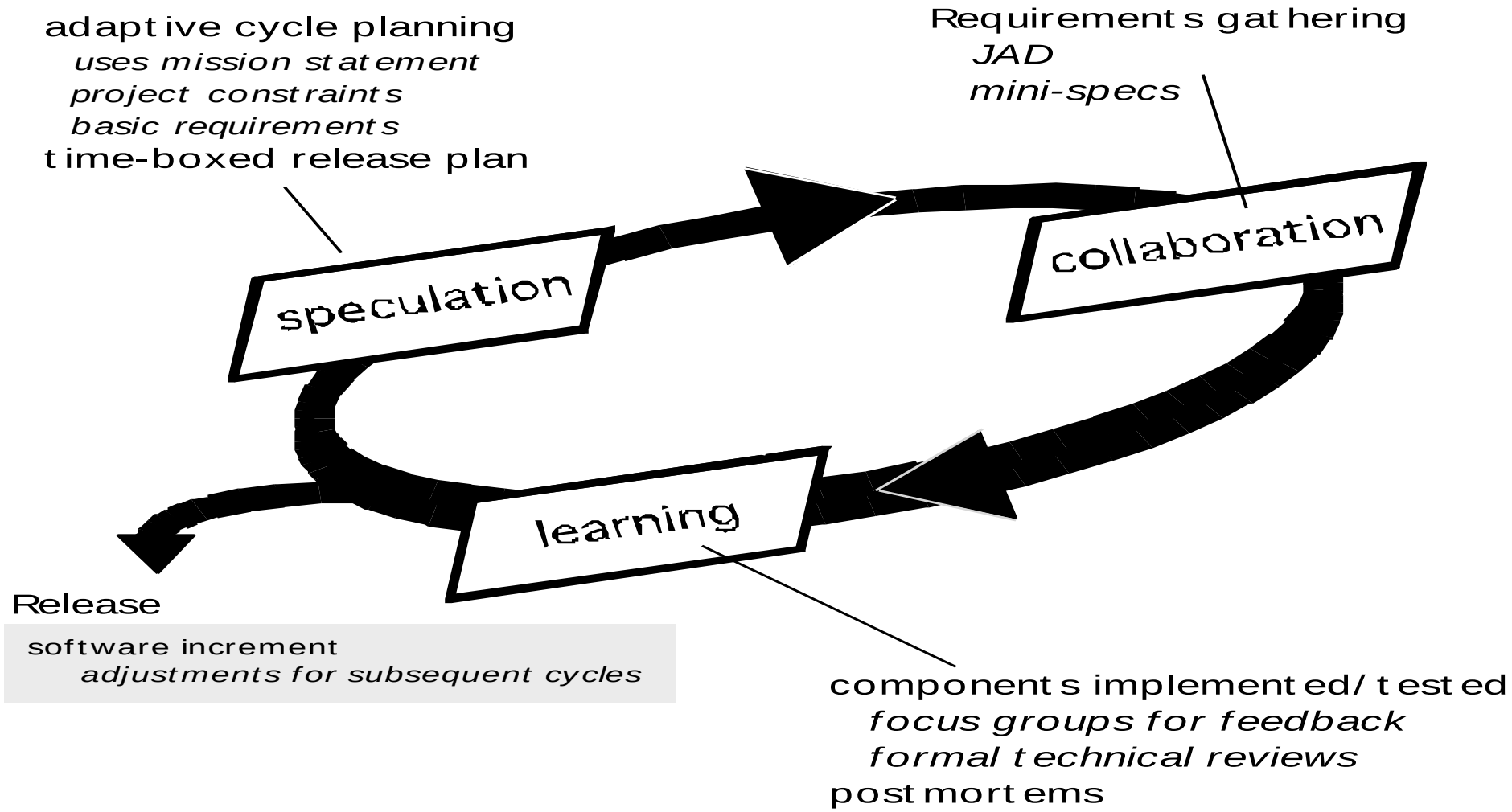
Three Phases of ASD

- 1. Speculation: project is initiated and adaptive cycle planning is conducted.
- Adaptive cycle planning uses project initiation information- the customer's mission statement, project constraints (e.g. delivery date), and basic requirements to define the set of release cycles (increments) that will be required for the project.
- Based on the information obtained at the completion of the first cycle, the plan is reviewed and adjusted so that planned work better fits the reality.

- 2. Collaboration are used to multiply their talent and creative output beyond absolute number (1+1>2). It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking.
  - It is a matter of trust. 1) criticize without animosity, 2) assist without resentments, 3) work as hard as or harder than they do. 4) have the skill set to contribute to the work at hand, 5) communicate problems or concerns in a way that leads to effective action.

- 3. Learning: As members of ASD team begin to develop the components, the emphasis is on “learning”. Highsmith argues that software developers often overestimate their own understanding of the technology, the process, and the project and that learning will help them to improve their level of real understanding.
- Learn in Three ways:
  - focus groups,
  - technical reviews (check code and design)
  - project postmortems (review after work).

Adaptive Software Development



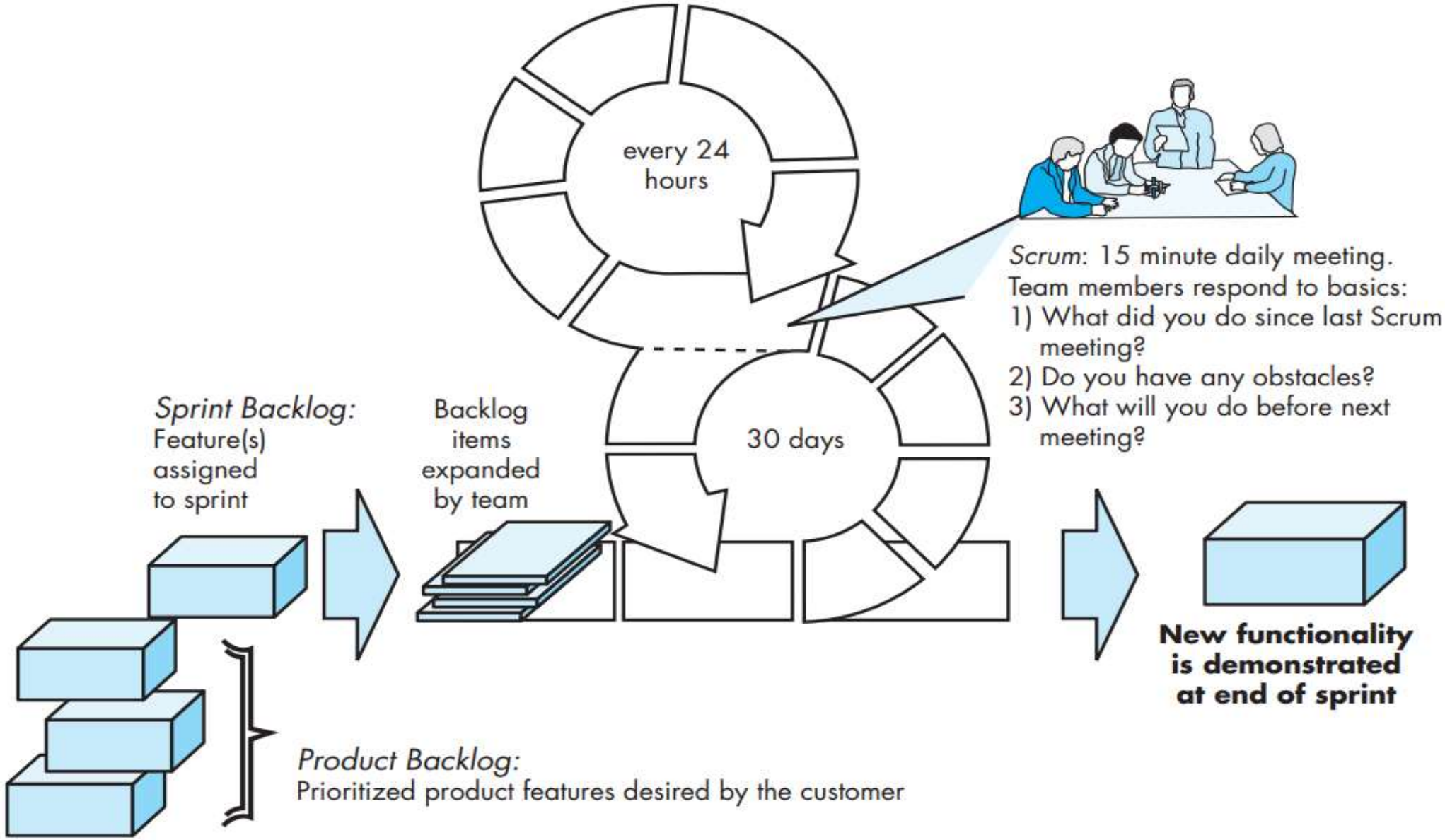
Scrum

- A software development method that was conceived by Jeff Sutherland and his development team in the early 1990s
- further development on the Scrum methods has been performed by Schwaber and Beedle .
- Scrum - the name is derived from an activity that occurs during a rugby match.

(A group of players forms around the ball and the teammates work together (sometimes violently!) to move the ball downfield.)

- Scrum is a subset of Agile and one of the most popular process frameworks for implementing Agile.
- It is an iterative development model often used to manage complex software and product development.
- Fixed-length iterations, called sprints lasting 30days long, allow the team to ship software on a regular cadence.
- At the end of each sprint, stakeholders and team members meet to plan next steps.

- Scrum—distinguishing features
  - Development work is partitioned into “packets”
  - Testing and documentation are on-going as the product is constructed
  - Work units occurs in “sprints” and is derived from a “backlog” of existing changing prioritized requirements
  - Changes are not introduced in sprints (short term but stable)
  - Meetings are very short (15 minutes daily)
  - What did you do since last meeting?
  - What obstacles are you encountering?
  - What do you plan to accomplish by next meeting?
  - “demos” are delivered to the customer with the time-box allocated. May not contain all functionalities. So customers can evaluate and give feedbacks.


every 24 hours
30 days
Scrum: 15 minute daily meeting.
Team members respond to basics:
1) What did you do since last Scrum meeting?
2) Do you have any obstacles?
3) What will you do before next meeting?
Sprint Backlog:
Feature(s) assigned to sprint
Backlog items expanded by team
New functionality is demonstrated at end of sprint
Product Backlog:
Prioritized product features desired by the customer

Dynamic Systems Development Method

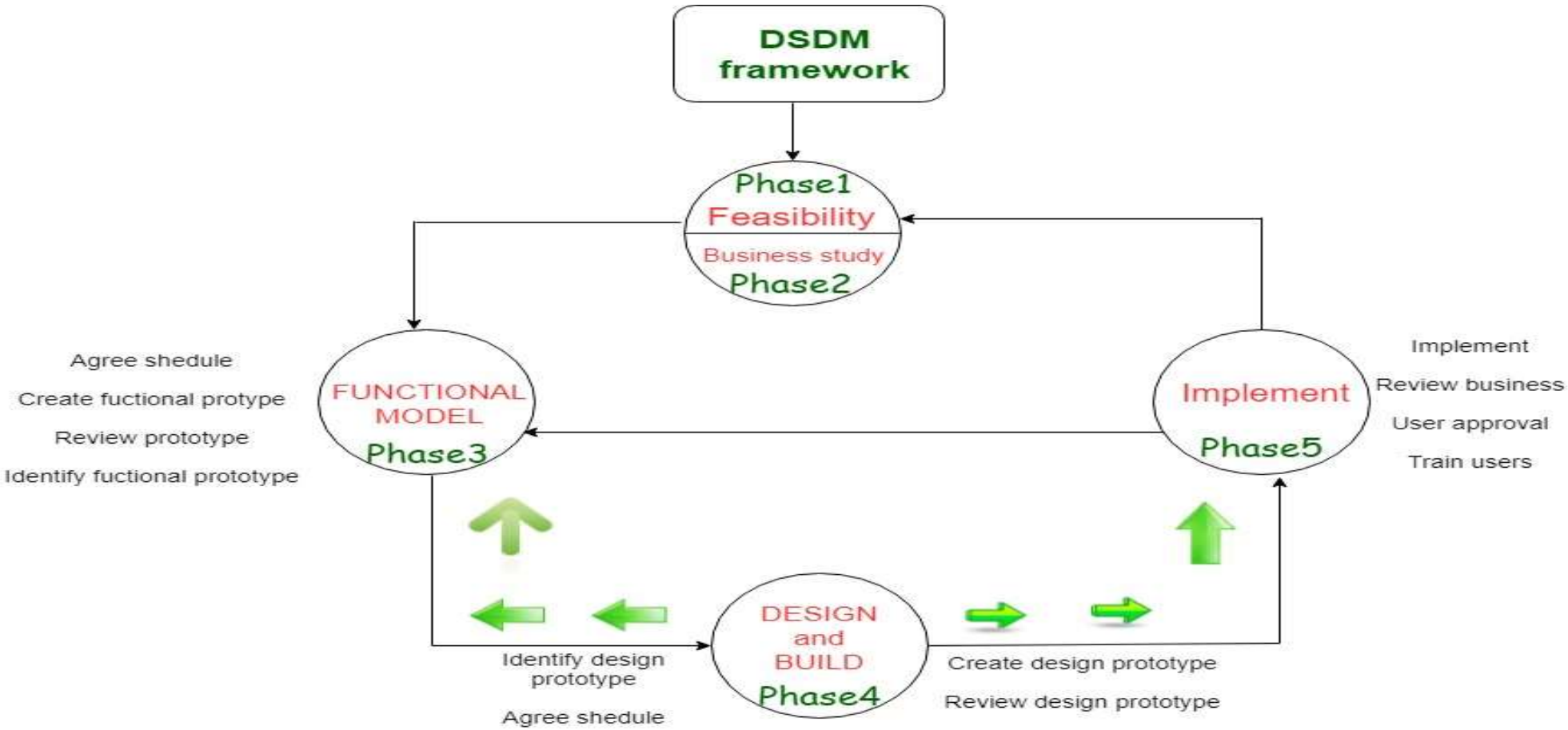
- It is an agile software development approach that provides a framework for building and maintaining systems which meet tight time constraints through the use of incremental prototyping in a controlled project environment.
- DSDM—distinguishing features
  - Similar in most respects to XP and/or ASD

- Dynamic System Development Method is approach to system development which as the name suggests develops the system dynamically.
- It is an associate degree agile code development approach that provides a framework for building and maintaining systems.
- An iterative & incremental approach that emphasizes continuous user/customer involvement.
- Goal-to deliver projects on time & on budget while adjusting for changing requirements along the way.

- DSDM is a methodology that prioritizes schedule and quality over functionality.
- It is Straight forward framework.
- Simple & Extendible

Principles of DSDM

- Eight guiding principles
  - ✓Active user involvement is imperative.
  - ✓DSDM teams must be empowered to make decisions.
  - ✓The focus is on frequent delivery of products.
  - ✓Fitness for business purpose is the essential criterion for acceptance of deliverables.
  - ✓Iterative and incremental development is necessary to converge on an accurate business solution.
  - ✓All changes during development are reversible.
  - ✓Requirements are baselined at a high level.
  - ✓Testing is integrated throughout the life-cycle.



Dynamic Systems Development Method life cycle

DSDM life cycle:

- **Feasibility Study:**
  It establishes the essential business necessities and constraints related to the applying to be designed then assesses whether or not the application could be a viable candidate for the DSDM method.
- **Business Study:**
  It establishes the use and knowledge necessities that may permit the applying to supply business value; additionally, it is the essential application design and identifies the maintainability necessities for the applying.

- **Functional Model Iteration:**
  It produces a collection of progressive prototypes that demonstrate practicality for the client.
  (Note: All DSDM prototypes are supposed to evolve into the deliverable application.) The intent throughout this unvarying cycle is to collect further necessities by eliciting feedback from users as they exercise the paradigm.

- **Design and Build Iteration:**
  It revisits prototypes designed throughout useful model iteration to make sure that everyone has been designed during a manner that may alter it to supply operational business price for finish users. In some cases, useful model iteration style and build iteration occur at the same time.

- **Implementation:**
  It places the newest code increment (an “operationalized” prototype) into the operational surroundings. It ought to be noted that:
  1. **(a)** the increment might not 100% complete or,
  2. **(b)** changes are also requested because the increment is placed into place. In either case, DSDM development work continues by returning to the useful model iteration activity.

Advantages

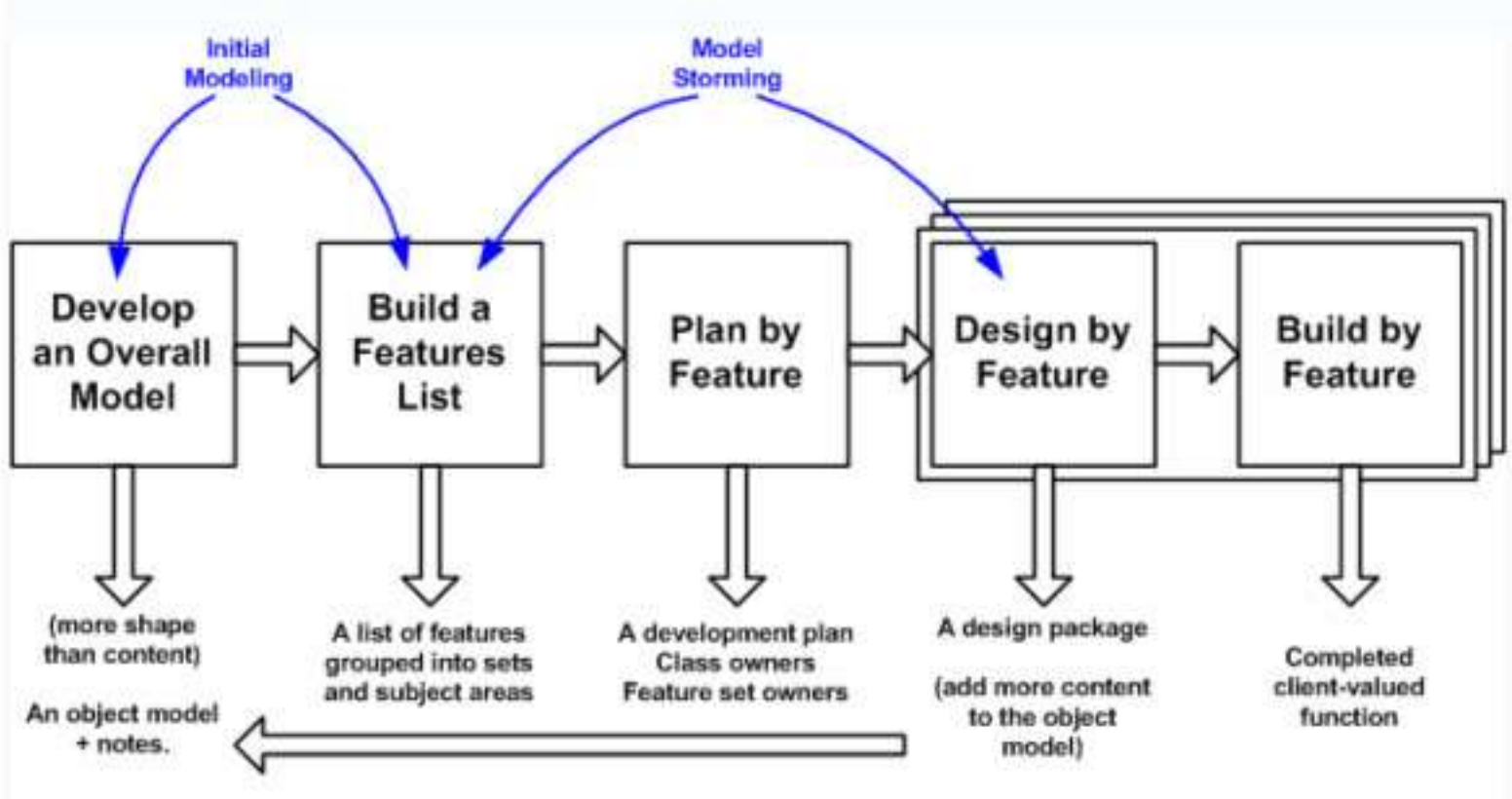
- Users are highly involved in the development.
- Basic functionality is delivered within very short time.
- Provides easy access for Developers to talk with end-users.
- Projects are delivered on time within a specific budget.

Disadvantages

- Sometimes its Costly
- May not suitable for small Organizations.
- Not suitable for one time projects
- Users and Developers both need to be trained.

Feature Driven Development

- Originally proposed by Peter Coad as an object-oriented software engineering process model.
- FDD—distinguishing features
  - Emphasis is on defining “features” which can be organized hierarchically.
    - a *feature* “is a client-valued function that can be implemented in two weeks or less.”
  - Uses a feature template
    - <action> the <result> <by | for | of | to> a(n) <object>
    - E.g. Add the product to shopping cart.
    - Display the technical-specifications of the product.
    - Store the shipping-information for the customer.
  - A features list is created and “plan by feature” is conducted
  - Design and construction merge in FDD


Initial Modeling
Model Storming
Develop an Overall Model
Build a Features List
Plan by Feature
Design by Feature
Build by Feature
(more shape than content)
An object model + notes.
A list of features grouped into sets and subject areas
A development plan
Class owners
Feature set owners
A design package
(add more content to the object model)
Completed client-valued function

Agile Modeling

- At “The Official Agile Modeling Site,” Scott Ambler [Amb02a] describes agile modeling (AM) in the following manner:
- Agile Modeling (AM) is a practice-based methodology for effective modeling and documentation of software-based systems.
- Simply put, Agile Modeling (AM) is a collection of values, principles, and practices for modeling software that can be applied on a software development project in an effective and light-weight manner. Agile models are more effective than traditional models because they are just barely good, they don’t have to be perfect.

Key Ideas:

- **Model with a purpose:** A developer who uses AM should have a specific goal in mind before creating the model. Once the goal for the model is identified, the type of notation to be used and level of detail required will be more obvious.
- **Use multiple models:** There are many different models and notations that can be used to describe software. AM suggests that to provide needed insight, each model should present a different aspect of the system and only those models that provide value to their intended audience should be used.
- **Travel light:**As software engineering work proceeds, keep only those models that will provide long-term value and discard the rest.

- **Content is more important than representation:** Modeling should impart information to its intended audience. A syntactically perfect model that imparts little useful content is not as valuable as a model with flawed notation that nevertheless provides valuable content for its audience.
- **Adapt locally:** The modeling approach should be adapted to the needs of the agile team.

  - A major segment of the software engineering community has adopted the Unified Modeling Language (UML) as the preferred method for representing analysis and design models.